מדריך התחלת עבודה – פרויקט ניהול פדגוגי

מדריך למפתחות · כל מה שצריך כדי להתחיל לעבוד על הפרויקט

לפני שמתחילות

בסוף המדריך הזה כל אחת תוכל להריץ את המערכת אצלה במחשב, להבין מה המסכים שלה, ולפתוח את ה-PR הראשון שלה. כדאי לעבור על השלבים לפי הסדר, ולסמן את רשימת הבדיקה שבסוף.

אנחנו בונות יחד **מערכת לניהול פדגוגי** של בית ספר: תלמידים, כיתות, ציונים, נוכחות ומערכת שעות. העבודה מתחלקת לצוותים, ולכל מפתחת יש **שני מסכים** שהיא בונה מקצה לקצה – מהממשק ועד השרת והבדיקות – לאורך חמישה ספרינטים.

הקוד נמצא בשלושה מאגרים (repositories) בארגון `ChatWithSchool` ב-GitHub:

מאגר	מה יש בו	ענף ראשי
`school-client`	צד הלקוח: React, TypeScript, Redux ו-Tailwind	`dev`
`school-server`	צד השרת: Node.js, TypeScript, Express ו-Mongoose מול MongoDB	`dev`
`docs`	האפיון, העיצוב, תרשימי הזרימה וחלוקת הצוותים. לקריאה בלבד	`main`

סדר השלבים: חשבונות ← התקנות ← הורדה מ-GitHub ← קריאת האפיון ← הרצה מקומית ← צורת העבודה.

שלב 1: חשבונות והזמנות

כל אחת עובדת עם האימייל האישי שלה במכללה (בתבנית `name@malot.org.il`). למייל הזה יגיעו הזמנות למערכות הפרויקט.

1. **GitHub** – אם אין לך חשבון, פתחי חשבון ב-github.com עם האימייל שלך, ושלחי למדריך את **שם המשתמש** שלך. אחרי שתגיע הזמנה לארגון `ChatWithSchool` – אשרי אותה (Accept invitation). בלי אישור ההזמנה לא תוכלי להוריד את המאגרים.
2. **Jira** – אשרי את ההזמנה ל-Jira של הצוות שלך. שם תראי את המשימות שלך.
3. **Figma** – אשרי את ההזמנה. יש לך הרשאת צפייה בלבד, וזה מספיק כדי לראות את עיצוב המסכים.
4. **מזהה מפתחת** – כל אחת מקבלת מהמדריך מזהה כמו `A-03` או `B-05`. לפי המזהה הזה מוצאים את שני המסכים שלך באפיון.

שלב 2: התקנות במחשב

צריך ארבע תוכנות. אחרי כל התקנה פותחים טרמינל חדש ומוודאים שהפקודה בעמודה האחרונה מחזירה מספר גרסה. (יש סיכוי שהכול מותקן כבר – רק צריך לוודא.)

תוכנה	למה צריך	איך בודקים
Git (מהאתר git-scm.com)	הורדת הקוד ועבודה עם ענפים. ב-Windows ההתקנה כוללת גם את Git Bash ואת Git Credential Manager	`git --version`
Node.js 22 LTS (מהאתר nodejs.org)	הרצת הקליינט והשרת. כוללת את `npm`	`node -v` `npm -v`
Docker Desktop (מהאתר docker.com)	הרצת מסד הנתונים MongoDB במחשב, בשביל השרת	`docker --version`
Visual Studio Code	עורך הקוד	–

הגדרה חד-פעמית של Git, עם השם והאימייל שלך (אותו אימייל שרשום ב-GitHub):

```
git config --global user.name "Your Name"
git config --global user.email "your-email@malot.org.il"
git config --global core.autocrlf input
```

בשורה הראשונה כותבים את השם שלך באנגלית. ההגדרה האחרונה שומרת על סופי שורות אחידים בקבצים, כדי שלא יופיעו שינויים מיותרים ב-PR בין מחשבי Windows לשרתי Linux.

תוספים מומלצים ל-VS Code: **ESLint**, **Prettier**, **Tailwind CSS IntelliSense** ו-**GitLens**.

שלב 3: הורדת המאגרים מ-GitHub

הורדה של מאגר נקראת **clone** (שכפול). מורידים פעם אחת את שלושת המאגרים לתיקייה אחת במחשב, ומשם והלאה רק מעדכנים.

1. צרי תיקיית עבודה, למשל `C:\projects\ChatWithSchool` (בלי רווחים ובלי עברית בנתיב – זה מונע תקלות בכלים).
2. פתחי בתוכה טרמינל: ב-VS Code דרך Terminal ← New Terminal, או Git Bash בלחיצה ימנית על התיקייה.
3. הריצי:

```
git clone https://github.com/ChatWithSchool/school-client.git
git clone https://github.com/ChatWithSchool/school-server.git
git clone https://github.com/ChatWithSchool/docs.git
```

4. בפעם הראשונה ייפתח חלון התחברות ל-GitHub. בחרי **Sign in with your browser** ואשרי. התוכנה Git Credential Manager שומרת את ההתחברות, ולא תצטרכי להתחבר שוב.

5. בסוף יהיו בתיקייה שלוש תיקיות: `school-client`, `school-server`, `docs`. פתחי כל אחת מהן ב-VS Code בנפרד (File ← Open Folder).

> **אם מופיעה שגיאה** `Repository not found` או `403` – בדרך כלל עוד לא אישרת את ההזמנה לארגון, או שאת מחוברת ב-GitHub עם חשבון אחר. בדקי בכתובת github.com/settings/organizations שהארגון מופיע אצלך.

שלב 4: הכרת האפיון במאגר docs

כל מה שצריך לדעת על המסכים שלך נמצא במאגר `docs`. אין צורך להתקין כלום: פותחים את הקובץ `index.html` בדפדפן (לחיצה כפולה), והוא מקשר לכל שאר הדפים. את הקבצים במאגר הזה **לא עורכים**.

מה יש בתיקייה:

תיקייה	מה יש בה
`spec/`	אתר האפיון: אינדקס המסכים, תוכנית העבודה, לוח המפתחות, עקרונות UX, ועמוד לכל מסך בתיקייה `spec/screens/`
`design-kit/`	מדריך העיצוב: צבעים, טיפוגרפיה, רכיבים והטוקנים של Tailwind
`flows/`	תרשים זרימת המסכים ותרשימי התהליכים המרכזיים
`teams/`	חלוקה לצוותים, תפקידים וניווט, ומטריצת ההרשאות

מה לקרוא, לפי הסדר:

1. **תוכנית העבודה** (`spec/plan.html`) – חמשת הספרינטים, מה עושים בכל אחד, והגדרת "הושלם" לכל שלב.
2. **לוח המפתחות** (`spec/tasks.html`) – חפשי את המזהה שלך (למשל `A-03`). שם רשומים שני המסכים שלך, האוספים (collections) שבאחריותך, והמשימות לכל ספרינט.
3. **עמוד המסך** (`spec/screens/SCR-XXX.html`) – לכל אחד משני המסכים. קראי בעיקר את "מה נבנה בגרסה 1" ואת "קריטריוני קבלה".
4. **עקרונות UX** (`spec/principles.html`) – כללים שחלים על כל המסכים: כיוון מימין לשמאל, טפסים, טבלאות, מצבי טעינה ושגיאה, נגישות.
5. **מדריך העיצוב** (`design-kit/design-guide.html`) – לפני שכותבים את הקליינט.
6. **חלוקה לצוותים** (`teams/team-split.html`) – בעיקר טבלת "תלויות בין צוותות", אם המסך שלך תלוי במסך של צוות אחר.

שלושה כללים לקריאת האפיון:

- כשיש סתירה בין התיאור המלא של המסך לבין **היקף v1-lite** – היקף v1-lite קובע. בונים רק את מה שנכלל בגרסה 1.
- מקור האמת לצורה של המסך הוא **Figma**. השרטוט בעמוד המסך הוא סכמה בלבד.
- בקליינט משתמשים רק בצבעים ובמרווחים מהטוקנים של Tailwind, לא בערכים שרירותיים.

שלב 5: הרצת הקליינט והשרת במחשב

בספרינט 1 עובדים על הקליינט עם **נתוני דמה** (Mock), ועל הסכמות וה-seed בשרת – הכול מקומית, בלי סביבת ענן. את הפקודות מריצים מתוך התיקייה של כל מאגר.

השרת (school-server)

לפני ההרצה ודאי שהתוכנה Docker Desktop פתוחה ופועלת.

```
cd school-server
npm install
cp .env.example .env
docker compose up -d
npm run seed
npm run dev
```

פקודה	מה היא עושה
`npm install`	מתקינה את הספריות. פעם אחת, ושוב כשמישהי מוסיפה ספרייה
`cp .env.example .env`	יוצרת קובץ הגדרות מקומי. ב-PowerShell כותבים `copy .env.example .env`. את הקובץ `.env` לא מעלים ל-GitHub
`docker compose up -d`	מפעילה את MongoDB ברקע
`npm run seed`	ממלאה נתוני הדגמה. אפשר להריץ שוב בלי כפילויות
`npm run dev`	מפעילה את השרת בכתובת `http://localhost:4000`

בדיקה שהשרת עובד: לפתוח בדפדפן את הכתובת `http://localhost:4000/api/health`. משתמשי ההדגמה ופרטי הכניסה שלהם מודפסים בטרמינל בסוף ההרצה של `npm run seed`.

הקליינט (school-client)

```
cd school-client
npm install
npm run dev
```

פותחים בדפדפן את הכתובת שמופיעה בטרמינל (בדרך כלל `http://localhost:5173`). הקליינט רץ עם נתוני דמה (`VITE_USE_MOCKS=true`), ולכן הוא עובד גם בלי שהשרת פועל. ההגדרה המדויקת מופיעה בקובץ README של הקליינט.

כדאי להיכנס עם כמה משתמשי הדגמה בתפקידים שונים (הנהלה, מזכירות, מורה, הורה) ולראות איך התפריט משתנה לפי ההרשאות.

דוגמאות ללמידה

לפני שכותבים קוד, עוברים על הדוגמה המלאה שהוכנה מראש:

- **בשרת:** התיקייה `src/modules/announcements` – מודול מלא עם הרשאות, ולידציה, עימוד, seed ובדיקות. ההסבר לכתיבת מודול חדש נמצא בקובץ `docs/new-module.md`.
- **בקליינט:** מסך ההדרכה בכתובת `/reference/announcements`, ותיקיית מסך עם הקבצים `index.tsx` ו-`SPEC.md`.

בדיקה לפני כל PR: npm run verify

בשני המאגרים יש פקודה אחת שבודקת הכול: TypeScript, ESLint והבדיקות. מריצים אותה עכשיו פעם אחת כדי לוודא שהסביבה תקינה, ומשם והלאה **לפני כל PR**. אם היא נכשלת – ה-PR לא ימוזג.

שלב 6: צורת העבודה

כל ספרינט נמשך שבוע, מסוף יום רביעי עד יום רביעי הבא. בכל ספרינט כל אחת עובדת על אותו שלב בשני המסכים שלה.

המחזור השבועי

מתי	מה עושות
סוף יום רביעי	מקבלות ב-Jira שתי משימות (שני המסכים, בשלב הנוכחי). קוראות את תיאור המשימה, את עמוד המסך באפיון ואת הפריים ב-Figma
עד יום שלישי	עובדות בענף משלך: פיתוח ובדיקה מקומית
יום שלישי	פותחות PR לענף `dev`, חברה עושה קוד ריוויו, ואחרי אישור ממזגות. בודקות ידנית לפי קריטריוני הקבלה
יום רביעי	מיזוג מ-`dev` ל-`stage` (על ידי ראשות הצוותים), וקבלת המשימות הבאות

השלבים לאורך הספרינטים

ספרינט	שלב	מה בונים
1	קליינט + סכמות ו-seed	המסך נראה כמו ב-Figma, על נתוני דמה. סכמות Mongoose ו-seed לאוספים שלך
2	סרבר א'	נקודות קצה לקריאה (GET) לפי החוזה בקובץ `openapi/openapi.yaml`
3	סרבר ב'	כתיבה, ולידציה, הרשאות ושגיאות, עם בדיקה בסיסית לכל endpoint
4	חיבור מקצה לקצה	הקליינט מול השרת האמיתי, בלי Mock, ובדיקה ידנית בסביבת `dev`
5	בדיקות אוטומטיות	בדיקות CI ירוקות וכיסוי של 70% לקוד שלך

ענפים

- הענף `stage` – מה שמוצג ונבדק. מתעדכן רק ביום רביעי.

- הענף `dev` – הענף המשותף. כל ה-PRים נכנסים אליו.
- הענף שלך – נפתח תמיד מ-`dev` עדכני, בתבנית `feature/<task-key>-<screen>-<stage>`. לדוגמה: `feature/PEDA-12-S01-client`.

את מפתח המשימה (למשל `PEDA-12`) מעתיקים מ-Jira כשהוא מופיע בשם הענף ובכותרת ה-PR, המערכת Jira מקשרת אליהם אוטומטית.

> **אסור לדחוף ישירות לענפים `dev` או `stage`.** הכול עובר דרך PR – הענפים האלה מוגנים, ו-GitHub ידחה דחיפה ישירה.

זרימת העבודה על משימה

1. ב-Jira: מעבירים את המשימה לסטטוס **In Progress**.
2. מעדכנים את `dev` ופותחים ממנו ענף חדש (הפקודות בפרק הבא).
3. עובדים בקומיטים קטנים. הודעת ה-commit באנגלית, קצרה, ומתחילה במזהה המסך, לדוגמה: `S01: add students table with search`.
4. מריצים `npm run verify` ומוודאים שהפקודה עוברת.
5. דוחפים את הענף ופותחים ב-GitHub בקשת **Pull Request** לענף `dev`. ממלאים את תבנית ה-PR, כולל קישור למשימה ב-Jira.
6. ב-Jira: מעבירים לסטטוס **In Review**, ושולחים קישור ל-PR בערוץ הקוד ריוויו עם שורה או שתיים של הסבר.
7. אחרי אישור ובדיקות ירוקות – ממזגים ל-`dev`, ובודקים ידנית שם. ב-Jira: סטטוס **In Dev**. אחרי המיזוג ל-`stage` המשימה עוברת לסטטוס **Done**.

כללים ל-PR ולקוד ריוויו

- כל PR קטן, עד כ-400 שורות, ומטפל במשימה אחת.
- חברה אחת לפחות מאשרת. שינוי בחלקים המשותפים (`models`, `permissions`, `middleware`, `seed`, `openapi`) דורש אישור של ראשת צוות.
- כל אחת עובדת בתיקייה משלה: בשרת – מודול משלה בתוך `src/modules/`, ובקליינט – תיקיית מסך משלה. כך כמעט אין התנגשויות.
- בקוד ריוויו בודקות: האם הקוד עושה מה שהמשימה מבקשת, האם יש בדיקות, האם ההרשאות נבדקות, והאם הקוד קריא.
- הערות בקוד ריוויו כותבות בעדינות ולעניין, בצורה של הצעה.
- שפה: קוד, שמות משתנים והודעות commit באנגלית. טקסטים בממשק, תיעוד והערות ב-PR – בעברית.

פקודות Git לעבודה יומיומית

התחלת משימה חדשה – תמיד מ-`dev` עדכני:

```
git switch dev
git pull
git switch -c feature/PEDA-12-S01-client
```

שמירת עבודה ודחיפה ל-GitHub:

```
git status
git add .
git commit -m "S01: add students table with search"
git push -u origin feature/PEDA-12-S01-client
```

אחרי הדחיפה הראשונה מספיק לכתוב `git push`. לפני `git add .` כדאי להסתכל בפלט של `git status` ולוודא שלא נכנסים קבצים מיותרים.

עדכון הענף שלך כשמשהו חדש נכנס ל-`dev`:

```
git switch dev
git pull
git switch feature/PEDA-12-S01-client
git merge dev
```

אם יש התנגשות (conflict), עורך הקוד VS Code מסמן את הקבצים. פותרים, שומרים, ואז מריצים `git add .` ו-`git commit`. לא בטוחה? שואלים לפני שממשיכים.

כללים חשובים

- לא מעלים ל-GitHub את הקובץ `.env`, סיסמאות או מפתחות. הקובץ `.gitignore` כבר מונע את רוב המקרים, אבל בודקים ב-`git status`.
- בשרת, כל route מתחיל בבדיקת הרשאה `authorize('<screen-id>', 'read' | 'write')`, וכל שאילתה מסוננת לפי `institutionId`.
- נקודות הקצה (endpoints) חייבות להתאים לחוזה בקובץ `openapi/openapi.yaml`. שינוי בחוזה – רק דרך ראשות הצוותים.
- את אוספי הליבה (`models`) לא משנים בלי אישור ראשת צוות. ה-seed שלך חייב לרוץ פעמיים בלי ליצור כפילויות.
- כל הנתונים במערכת הם נתוני הדגמה בדיוניים.

שאלות ועזרה

- קודם מנסים לבד **20 דקות**.
- שואלים בערוץ הצוות ולא בהודעה פרטית, כדי שכולן ילמדו מהתשובה.
- כותבים: מה ניסיתי, מה קרה, ומה ציפיתי שיקרה. מצרפים צילום מסך או שורת שגיאה, ואת מפתח המשימה (למשל `PEDA-12`).
- משימה שמחכה לצוות אחר מסמנים בתווית `blocked` ב-Jira, ומעדכנים את ראשת הצוות.

רשימת בדיקה – לסמן עד תחילת העבודה

- [ ] אישרתי את ההזמנות ל-GitHub, ל-Jira ול-Figma
- [ ] התקנתי Git, Node.js 22, Docker Desktop ו-VS Code
- [ ] הגדרתי שם ואימייל ב-Git
- [ ] הורדתי את שלושת המאגרים: `school-client`, `school-server`, `docs`
- [ ] השרת עולה: `npm run verify`, `docker compose up -d`, `npm run seed`, `npm run dev`
- [ ] הקליינט עולה: `npm install`, `npm run verify`, `npm run dev`, ונכנסתי עם משתמש הדגמה
- [ ] מצאתי במאגר `docs` את שני המסכים שלי וקראתי את עמודי המסכים
- [ ] אני רואה את המשימות שלי ב-Jira ואת הפריימים שלי ב-Figma
- [ ] עברתי על מודול הדוגמה `announcements` בשרת ובקליינט